Project File

Web Programming using

Python and JavaScript

Name: Aayush

Course: B-Tech CSE Core

Year: 2nd Year, 4th Semester

Roll No: 2401010051

Submitted To:

Ms. Mansi Kajal

EXPERIMENT – I

❖ **Aim :**

Creating a **Personal Portfolio Web Application** using **HTML, CSS, and JavaScript**.

❖ **Tasks :**

- **Project Setup & Introduction** – Create project folder and basic linked files.
- **HTML Structure** – Build portfolio sections using semantic HTML.
- **Styling with CSS** – Apply layout, colors, spacing, and neat design.
- **Projects & Skills Display** – Show skills and projects clearly on the page.
- **JavaScript Interactivity** – Add basic interactive functionality using JavaScript.
- **Contact Form (Bonus)** – Create a form with user submission feedback.

❖ **Tech Stack Used :**

- **HTML** , **CSS** and **JavaScript**

❖ Code :

➤ **HTML CODE :**

```
<!--  -->
<!-- Name:Pranjal Dutta -->
Project: Personal Portfolio -->
<!--  -->
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Pranjal Portfolio</title>
  <link rel="stylesheet" href="portfolio.css">
```

```
</head>

<body>

<header>
  <h1>Pranjal Dutta</h1>
  <p>I am learning Web Development</p>
</header>

<nav>
  <a href="#skills">Skills</a>
  <a href="#projects">Projects</a>
  <a href="#contact">Contact</a>
</nav>

<section class="welcome">
  <h2>Welcome</h2>
  <p>This is my beginner portfolio website.</p>
  <button onclick="showMessage()">Click Me</button>
</section>

<section id="skills">
  <h2>
    My Skills
    <button onclick="toggleSection('skillsContent')">Hide / Show</button>
  </h2>
  <ul id="skillsContent">
    <li>HTML</li>
    <li>CSS</li>
    <li>JavaScript (Beginner)</li>
  </ul>
```

```
</section>

<section id="projects">
  <h2>
    My Projects
    <button onclick="toggleSection('projectsContent')">Hide / Show</button>
  </h2>

  <div id="projectsContent">
    <div class="card">
      <h3>Portfolio Website</h3>
      <p>This is my personal portfolio made using HTML, CSS and JavaScript.</p>
    </div>

    <div class="card">
      <h3>Simple HTML Page</h3>
      <p>A basic web page created using only HTML.</p>
    </div>
  </div>
</section>

<section id="contact">
  <h2>Contact Me</h2>

  <form onsubmit="submitForm(); return false;">
    <label>Name:</label>
    <input type="text" required>

    <label>Email:</label>
    <input type="email" required>
```

```
        <label>Message:</label>
        <textarea rows="4" required></textarea>

        <button type="submit">Submit</button>
    </form>

    <p id="result"></p>
</section>
<footer>
    <p>© 2026 Pranjal</p>
</footer>

<button id="scrollBtn" onclick="scrollToTop()">⬆ </button>

<script src="portfolio.js"></script>
</body>
</html>
```

➤ **CSS CODE :**

```
@import url('https://fonts.googleapis.com/css2?family=Poppins:wght@300;400;600&display=swap');

* {
    box-sizing: border-box;
}

body {
    margin: 0;
    font-family: 'Poppins', sans-serif;
    background: linear-gradient(to right, #fbc2eb, #a6c1ee);
}
```

```
header {
   background: blue ;
   color: white;
   padding: 40px 20px;
   text-align: center;
}

header h1 {
   margin: 0;
   font-size: 38px;
}

header p {
   font-size: 18px;
   opacity: 0.9;
}

nav {
   background-color: #ffffff;
   padding: 12px;
   text-align: center;
   box-shadow: 0 4px 10px rgba(0,0,0,0.1);
}

nav a {
   color: red;
   text-decoration: none;
   margin: 0 15px;
   font-weight: 600;
}
```

```
nav a:hover {
    color: pink;
}

section {
    background-color: w;
    max-width: 900px;
    margin: 30px auto;
    padding: 25px;
    border-radius: 12px;
    box-shadow: 0 10px 25px rgba(0,0,0,0.15);
}

.welcome {
    text-align: center;
}

.welcome h2 {
    font-size: 28px;
}

button {
    background: linear-gradient(to right, #6a11cb, #2575fc);
    color: white;
    border: none;
    padding: 10px 18px;
    border-radius: 25px;
    cursor: pointer;
    font-size: 14px;
    transition: 0.3s;
```

```
}
button:hover {
   transform: scale(1.05);
}
.card {
   background: #f9f9f9;
   border-radius: 10px;
   padding: 20px;
   margin: 15px 0;
   transition: 0.3s;
}
.card:hover {
   transform: translateY(-5px);
   box-shadow: 0 8px 20px rgba(0,0,0,0.15);
}

ul {
   list-style: none;
   padding: 0;
}
ul li {
   background: #eef3ff;
   margin: 8px 0;
   padding: 10px;
   border-radius: 6px;
   font-weight: 500;
}
form {
   margin-top: 15px;
}
input, textarea {
```

```
    width: 100%;
    padding: 10px;
    margin: 8px 0;
    border-radius: 8px;
    border: 1px solid #ccc;
    font-family: 'Poppins', sans-serif;
}
input:focus, textarea:focus {
    outline: none;
    border-color: #2575fc;
}
#contact a {
    display: inline-block;
    margin: 10px 10px 0 0;
    text-decoration: none;
    font-weight: 600;
    color: #2575fc;
}

#contact a:hover {
    color: #6a11cb;
}

footer {
    background-color: #111;
    color: white;
    text-align: center;
    padding: 15px;
    margin-top: 40px;
}
#scrollBtn {
```

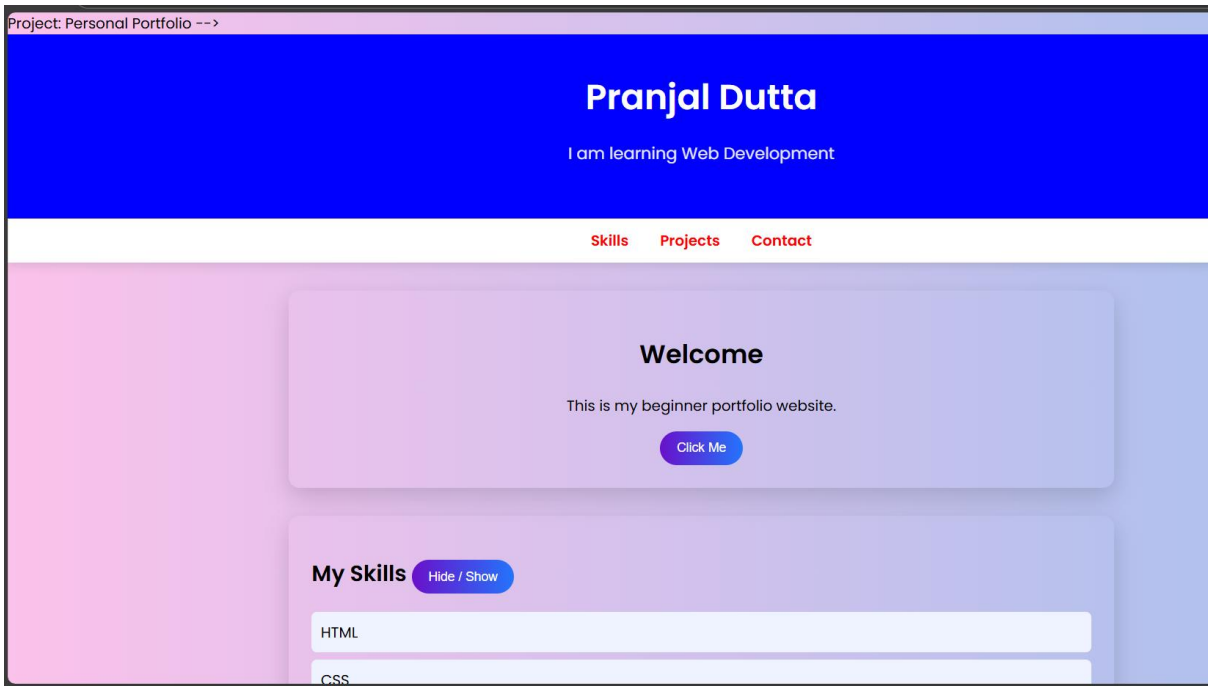
```
    position: fixed;
    bottom: 25px;
    right: 25px;
    display: none;
    background: linear-gradient(to right, #6a11cb, #2575fc);
    color: white;
    border-radius: 50%;
    border: none;
    padding: 12px 16px;
    font-size: 18px;
    box-shadow: 0 8px 15px rgba(0,0,0,0.3);
}
```

➤ **JS CODE :**

```
function showMessage() {
    alert("Welcome to my portfolio website!");
}
function submitForm() {
    document.getElementById("result").innerText =
        "Thank you! Your message has been submitted.";
}
function toggleSection(id) {
    const section = document.getElementById(id);
    if (section.style.display === "none") {
        section.style.display = "block";
    } else {
        section.style.display = "none";
    }
}
window.onscroll = function () {
    const btn = document.getElementById("scrollBtn");
```

```
  if (document.documentElement.scrollTop > 200) {
    btn.style.display = "block";
  } else {
    btn.style.display = "none";
  }
};
function scrollToTop() {
  window.scrollTo({ top: 0, behavior: "smooth" });
}
```

➤ **Output:**



JavaScript (Beginner)

My Projects

Hide / Show

Portfolio Website

This is my personal portfolio made using HTML, CSS and JavaScript.

Simple HTML Page

A basic web page created using only HTML.

Contact Me

Name:

Email:

Message:

Submit

© 2026 Pranjal

EXPERIMENT – II

❖ **Aim :**

Creating a **Company Landing Page** using **HTML and CSS** to display company information, services, testimonials, and contact details.

❖ **Tasks :**

- **Project Setup & Introduction** – Create project folder and basic HTML and CSS files.
- **HTML Structure** – Build landing page sections using semantic HTML tags.
- **Styling with CSS** – Apply layout, colors, fonts, spacing, and borders for a professional design.
- **Services Section Design** – Display services with service name and short description.
- **Testimonials Section** – Add customer feedback and style it neatly.
- **Contact Section (Bonus)** – Create contact information and a simple contact form.

❖ **Tech Stack Used :**

- **HTML , CSS and Javascript**

❖ **CODES :**

➤ **HTML CODE :**

```
<!--  -->

<!-- Name: Pranjal

Project: PranjalSolutions -->

<!--  -->

<!DOCTYPE html>

<html lang="en">

<head>

   <meta charset="UTF-8">

   <title>Company Landing Page</title>

   <link rel="stylesheet" href="style.css">

</head><wbr>


</wbr>
```

```
<body>

  <header>
    <h1>Pranjal Solutions</h1>
    <p>Innovating Technology for a Better Tomorrow</p>
  </header>

  <section>
    <h2>About Us</h2>
    <p>
      Pranjal Solutions provides modern and reliable
      technology services for businesses and individuals.
    </p>
  </section>

  <section>
    <h2>Our Services</h2>

    <div class="service-card">
      <h3>Web Development</h3>
    <p></p>We create simple and responsive websites.</p>
    </div>

    <div class="service-card">
      <h3>App Development</h3>
    <p></p>We develop easy-to-use mobile applications.</p>
    </div>

    <div class="service-card">
      <h3>Technical Support</h3>
```

```
    <p></p>We provide 24/7 technical assistance.</p>
    </div>
</section>

<section>
    <h2>Testimonials</h2>

    <div class="testimonial">
        <p>"Great service and friendly team!"</p>
    <h4></h4>-Pranjal</h4>
    </div>

    <div class="testimonial">
        <p>"Very professional and reliable company."</p>
    <h4></h4>-utkarsh</h4>
    </div>
</section>

<section>
    <h2>Contact Us</h2>

    <p>Email: Pranjal@Solutions.com</p>
    <p>Phone: 9876543210</p>
    <p>Address: New Delhi, India</p>

    <form onsubmit="showMessage(); return false;">
        <input type="text" placeholder="Name" required><br><br>
        <input type="email" placeholder="Email" required><br><br>
        <textarea rows="4" placeholder="Message" required></textarea><br><br>
        <button type="submit">Send</button>
    </form>
```

```
    </section>
    <footer>
        <p>@ Pranjal Solutions</p>
    </footer>

    <script src="script.js"></script>

</body>
</html>
```

➤CSS CODE :

```
@import url('https://fonts.googleapis.com/css2?family=Montserrat:wght@400;600;700&display=swap');

* {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
    font-family: 'Montserrat', sans-serif;
}

body {
    background: #0f172a;
    color: #e2e8f0;
    line-height: 1.6;
}

/* HEADER */
header {
    background: rgba(255, 255, 255, 0.05);
    backdrop-filter: blur(10px);
    padding: 70px 20px;
    text-align: center;
```

```
    border-bottom: 1px solid rgba(255,255,255,0.1);
}

header h1 {
    font-size: 48px;
    font-weight: 700;
    color: #38bdf8;
    letter-spacing: 1px;
}

header p {
    margin-top: 15px;
    font-size: 18px;
    color: #cbd5e1;
}
/* SECTION */
section {
    max-width: 1100px;
    margin: 50px auto;
    padding: 40px;
    background: rgba(255, 255, 255, 0.05);
    border-radius: 20px;
    backdrop-filter: blur(15px);
    border: 1px solid rgba(255,255,255,0.1);
}
section h2 {
    text-align: center;
    margin-bottom: 35px;
    font-size: 28px;
    color: #38bdf8;
    text-transform: uppercase;
    letter-spacing: 2px;
}
```

```
/* SERVICE CARDS */
.service-card {
    background: rgba(255, 255, 255, 0.08);
    padding: 25px;
    border-radius: 15px;
    margin-bottom: 20px;
    transition: 0.4s ease;
    border: 1px solid rgba(255,255,255,0.05);
}
.service-card h3 {
    margin-bottom: 10px;
    color: #f1f5f9;
}

.service-card p {
    color: #cbd5e1;
}

.service-card:hover {
    transform: scale(1.03);
    background: rgba(56, 189, 248, 0.15);
    box-shadow: 0 15px 25px rgba(0,0,0,0.4);
}
/* TESTIMONIAL */
.testimonial {
    background: rgba(255, 255, 255, 0.06);
    padding: 20px;
    border-radius: 15px;
    margin-bottom: 20px;
    border-left: 4px solid #38bdf8;
}
.testimonial p {
```

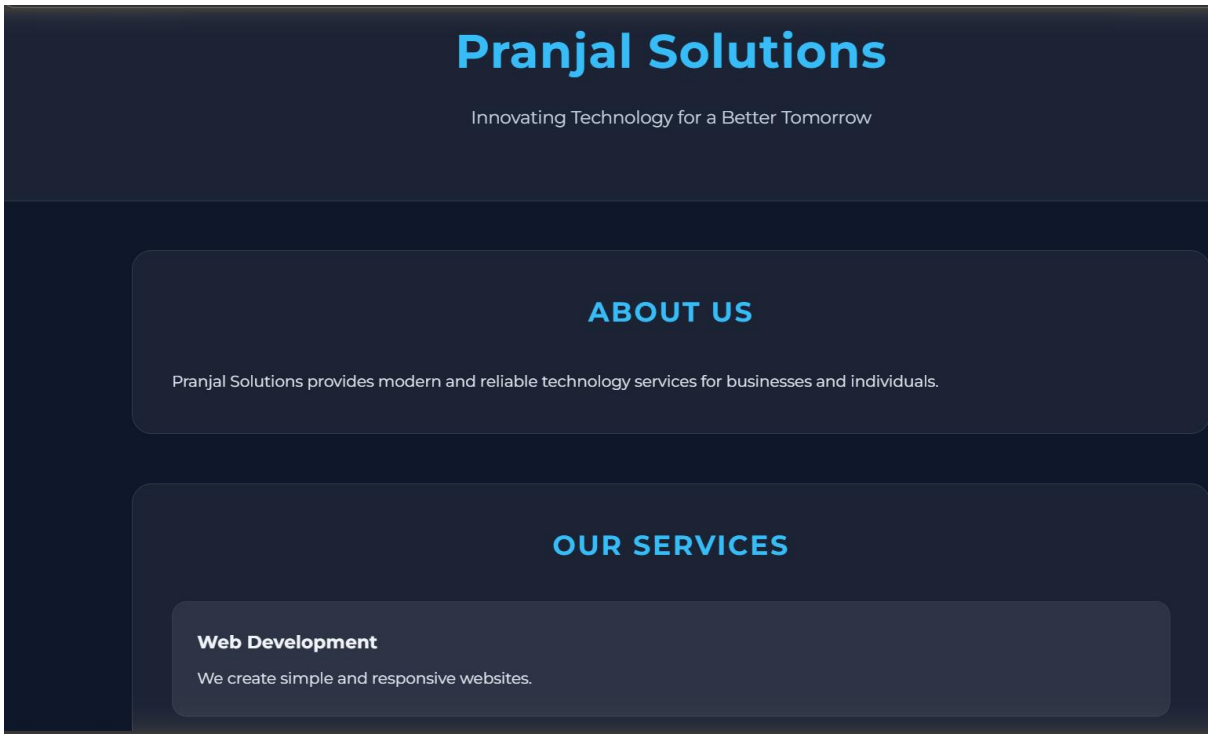
```
    font-style: italic;
    color: #e2e8f0;
}

.testimonial h4 {
    text-align: right;
    margin-top: 10px;
    color: #94a3b8;
}

/* FORM */
form {
    max-width: 500px;
    margin: 30px auto 0;
}
input, textarea {
    width: 100%;
    padding: 14px;
    margin-bottom: 15px;
    border-radius: 12px;
    border: none;
    background: rgba(255,255,255,0.08);
    color: #fff;
    font-size: 14px;
}

input:focus, textarea:focus {
    outline: none;
    background: rgba(56,189,248,0.15);
    border: 1px solid #38bdf8;
}
button {
    width: 100%;
```

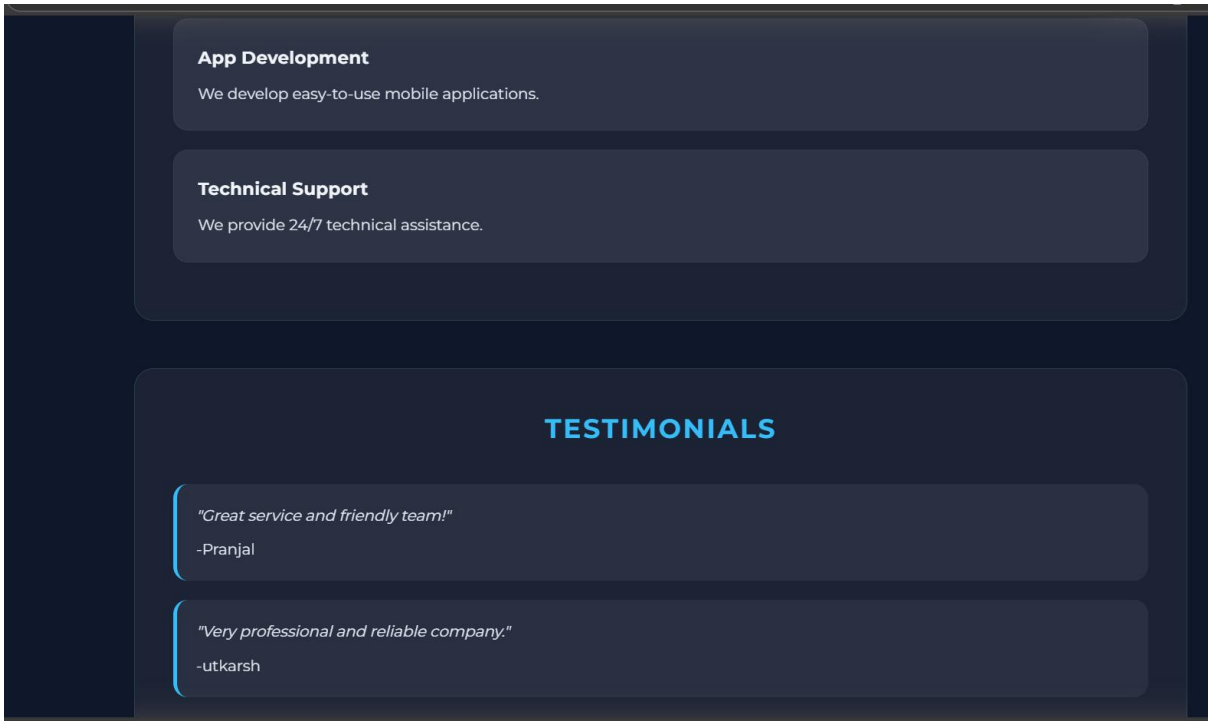
```
    padding: 14px;
    border-radius: 30px;
    border: none;
    background: linear-gradient(90deg, #38bdf8, #6366f1);
    color: white;
    font-size: 16px;
    cursor: pointer;
    transition: 0.3s ease;
}
button:hover {
    transform: translateY(-3px);
    box-shadow: 0 10px 20px rgba(0,0,0,0.5);
}

/* FOOTER */
footer {
    text-align: center;
    padding: 25px;
    font-size: 14px;
    color: #94a3b8;
}

/* RESPONSIVE */
@media (max-width: 768px) {
    header h1 {
        font-size: 36px;
    }
    section {
        margin: 25px 15px;
        padding: 25px;
    }
}
```

➤ **JS CODE :**

```
function showMessage() {
   alert("Thank you for contacting TechNova Solutions!");
}
```

❖ **OUTPUT :**





CONTACT US

Email: Pranjal@Solutions.com
Phone: 9876543210
Address: New Delhi, India

Name

Email

Message

Send

EXPERIMENT – 3

❖ **Aim :**
To design and develop an **Interactive Quiz Web Application** using **HTML, CSS, and JavaScript**.

❖ **Tasks :**
• **Project Setup & Introduction** – Create the project folder and link HTML, CSS, and JavaScript files properly.
• **Quiz Interface Design** – Design quiz layout with questions, options, and navigation buttons using semantic HTML.
• **Styling with CSS** – Apply colors, spacing, alignment, and layout to improve user experience.
• **Quiz Logic Implementation** – Store questions and answers in JavaScript and handle user input.
• **Score Tracking & Navigation** – Move between questions and calculate the score dynamically.

❖ **Tech Stack Used :**
• **HTML** •**CSS** • **JavaScript**

❖ Html Code :

```
<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <title>Interactive Quiz</title>

    <link rel="stylesheet" href="style.css">

</head>

<body>

<header>

    <h1>Interactive Quiz</h1>

    <p>Test your Web Development knowledge</p>

</header>


<main class="quiz-container">
```

```html
    <section id="intro">
        <h2>Quiz Introduction</h2>
        <p>This quiz contains multiple-choice questions. Click Start to begin.</p>
        <button onclick="startQuiz()">Start Quiz</button>
    </section>
    <section id="quiz" class="hidden">
        <h2 id="question"></h2>
        <div id="options"></div>
        <button onclick="nextQuestion()">Next</button>
    </section>
    <section id="result" class="hidden">
        <h2>Your Result</h2>
        <p id="scoreText"></p>
        <p id="feedback"></p>
        <button onclick="restartQuiz()">Restart Quiz</button>
    </section>
</main>
<script src="script.js"></script>
</body>
</html>
```

➤ **CSS CODE :**

```css
* {
    box-sizing: border-box;
    margin: 0;
    padding: 0;
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
}
body {
    background: radial-gradient(circle at top, #1e1e2f, #12121c);
```

```
    color: #ffffff;
    min-height: 100vh;
}
/* HEADER */
header {
    background: linear-gradient(90deg, #ff416c, #ff4b2b);
    padding: 25px;
    text-align: center;
    font-size: 22px;
    font-weight: bold;
    letter-spacing: 1px;
    box-shadow: 0 4px 15px rgba(0,0,0,0.4);
}
/* QUIZ CONTAINER */
.quiz-container {
    background: #1f1f2e;
    max-width: 650px;
    margin: 50px auto;
    padding: 30px;
    border-radius: 15px;
    box-shadow: 0 20px 40px rgba(0,0,0,0.6);
    border: 1px solid rgba(255,255,255,0.05);
}
/* OPTIONS */
.option {
    background: #2c2c3e;
    padding: 12px;
    margin: 12px 0;
    border-radius: 8px;
    cursor: pointer;
    transition: 0.3s ease;
```

```
    border: 1px solid transparent;
}
.option:hover {
    background: #3a3a55;
    border: 1px solid #ff416c;
    transform: translateX(5px);
}
/* BUTTON */
button {
    background: linear-gradient(90deg, #00c6ff, #0072ff);
    color: white;
    border: none;
    padding: 12px 20px;
    margin-top: 20px;
    border-radius: 30px;
    cursor: pointer;
    font-size: 14px;
    transition: 0.3s ease;
}
button:hover {
    transform: scale(1.05);
    box-shadow: 0 8px 20px rgba(0, 114, 255, 0.5);
}
/* HIDDEN CLASS */
.hidden {
    display: none;
}
/* FOOTER */
footer {
    text-align: center;
    padding: 15px;
```

```
    background: #0f0f17;
    color: #bbb;
    margin-top: 40px;
    font-size: 14px;
}
```

➤ **JS CODE :**

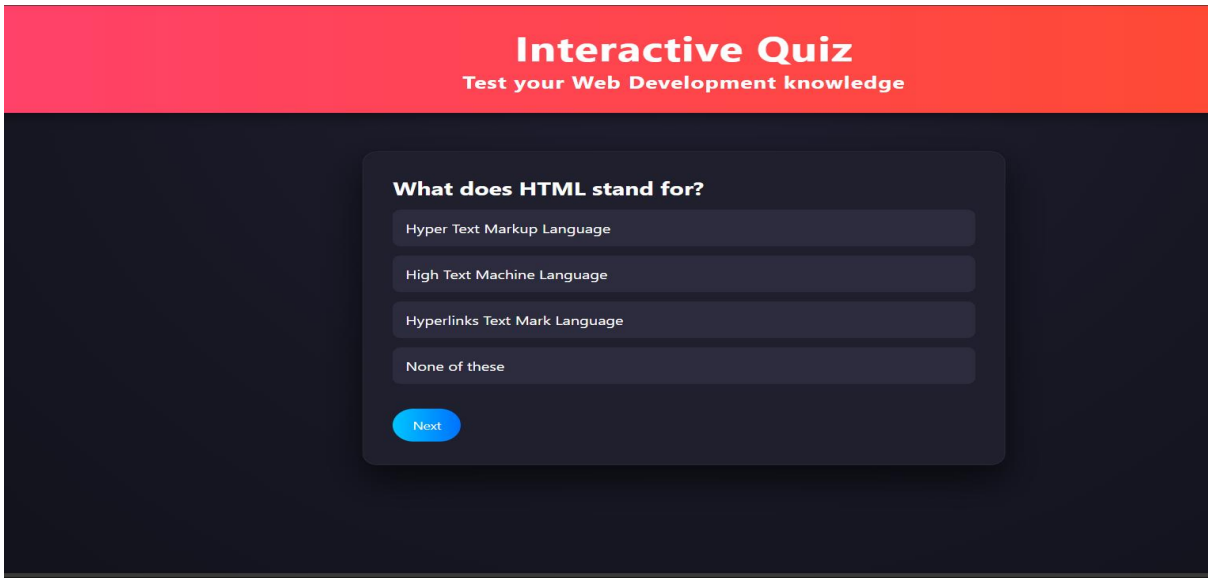
```
const quizData = [
    {
        question: "What does HTML stand for?",
        options: [
            "Hyper Text Markup Language",
            "High Text Machine Language",
            "Hyperlinks Text Mark Language",
            "None of these"
        ],
        correct: 0
    },
    {
        question: "Which language is used for styling web pages?",
        options: ["HTML", "CSS", "Java", "Python"],
        correct: 1
    },
    {
        question: "Which is used for web page interactivity?",
        options: ["HTML", "CSS", "JavaScript", "SQL"],
        correct: 2
    }
];

let currentQuestion = 0;
let score = 0;
```

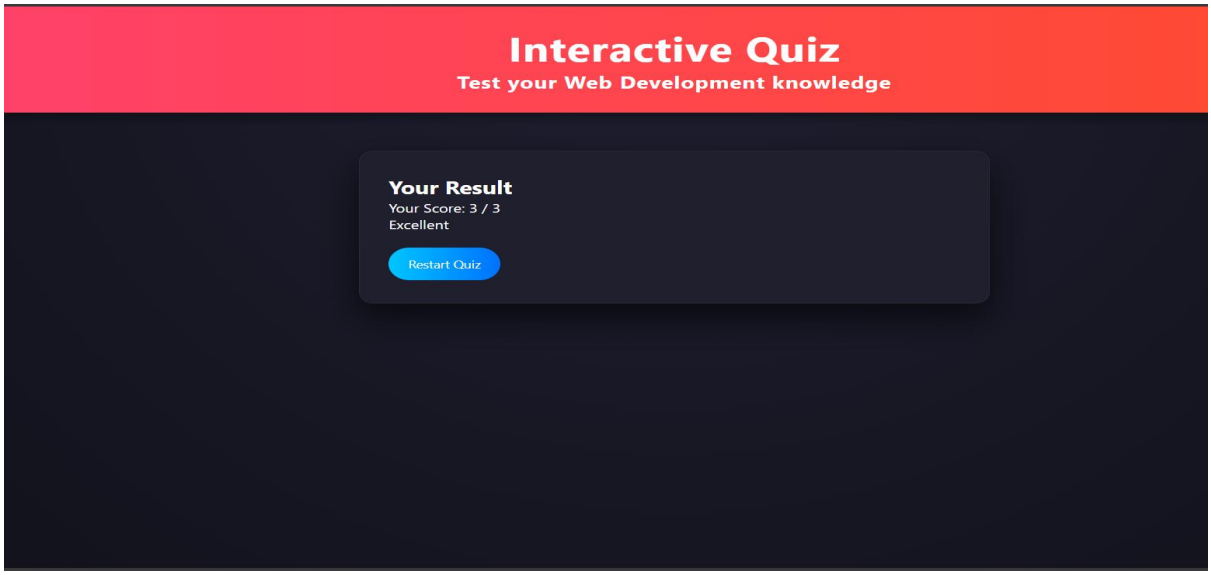
```
function startQuiz() {
    document.getElementById("intro").classList.add("hidden");
    document.getElementById("quiz").classList.remove("hidden");
    loadQuestion();
}

function loadQuestion() {
    const q = quizData[currentQuestion];
    document.getElementById("question").innerText = q.question;

    const optionsDiv = document.getElementById("options");
    optionsDiv.innerHTML = "";

    q.options.forEach((option, index) => {
        const div = document.createElement("div");
        div.className = "option";
        div.innerText = option;
        div.onclick = () => selectAnswer(index);
        optionsDiv.appendChild(div);
    });
}

function selectAnswer(selected) {
    if (selected === quizData[currentQuestion].correct) {
        score++;
    }
    nextQuestion();
}

function nextQuestion() {
```

```
    currentQuestion++;
    if (currentQuestion < quizData.length) {
        loadQuestion();
    } else {
        showResult();
    }
}

function showResult() {
    document.getElementById("quiz").classList.add("hidden");
    document.getElementById("result").classList.remove("hidden");

    document.getElementById("scoreText").innerText =
        `Your Score: ${score} / ${quizData.length}`;

    let feedback = "Try Again";
    if (score === quizData.length) feedback = "Excellent";
    else if (score >= 2) feedback = "Good";

    document.getElementById("feedback").innerText = feedback;
}

function restartQuiz() {
    currentQuestion = 0;
    score = 0;
    document.getElementById("result").classList.add("hidden");
    document.getElementById("intro").classList.remove("hidden");
}
```

➤ Output :

Interactive Quiz
Test your Web Development knowledge
Quiz Introduction
This quiz contains multiple-choice questions. Click Start to begin.
Start Quiz


Interactive Quiz
Test your Web Development knowledge
What does HTML stand for?
Hyper Text Markup Language
High Text Machine Language
Hyperlinks Text Mark Language
None of these
Next


Interactive Quiz
Test your Web Development knowledge
Your Result
Your Score: 3 / 3
Excellent
Restart Quiz

Experiment : 4

❖ **Aim :**
To design and develop a **Dynamic To-Do List Web Application** using **HTML, CSS, and JavaScript** that allows users to add, edit, delete, and manage tasks interactively.

❖ **Tasks :**

• **Project Setup & Introduction** – Create a project folder named **dynamic_todo_list** and add linked HTML, CSS, and JavaScript files with a short description of the application.

• **To-Do List Interface Design** – Create an input field to enter tasks, an **Add Task** button, and display tasks in a list format using HTML.

• **Styling with CSS** – Apply CSS styling to input fields, buttons, and task list to ensure proper alignment, spacing, and visual clarity.

• **Add & Display Tasks Using JavaScript** – Capture user input, add tasks dynamically to the list, and prevent empty tasks from being added.

• **Edit & Delete Task Features** – Provide **Edit** and **Delete** buttons for each task to allow modification and removal of tasks dynamically.

• **Task Completion Status (Bonus)** – Add a checkbox or toggle to mark tasks as completed and visually indicate completed tasks using a strikethrough effect.

❖ **Tech Stack Used :**
• **HTML** • **CSS** • **JavaScript**

❖ Html Code :

```
<!DOCTYPE html>
<html lang="en">
<head>

    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Dynamic To-Do List</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>
```

```
    <div class="container">
        <!-- Header Section -->
        <header class="header">
            <h1>My To-Do List</h1>
            <p class="description">Stay organized and productive. Add, edit, and track
your daily tasks.</p>
        </header>

        <!-- Input Section -->
        <div class="input-section">
            <input
                type="text"
                id="taskInput"
                class="task-input"
                placeholder="Enter a new task..."
                autocomplete="off"
            >
            <input
                type="datetime-local"
                id="dueInput"
                class="due-input"
                title="Due date and time"
            >
            <button id="addBtn" class="add-btn">Add Task</button>
        </div>

        <!-- Task Counter Section -->
        <div class="task-counter">
```

```
        <span class="counter-item">
            <strong>Total:</strong> <span id="totalTasks">0</span>
        </span>
        <span class="counter-item">
            <strong>Completed:</strong> <span id="completedTasks">0</span>
        </span>
        <span class="counter-item">
            <strong>Pending:</strong> <span id="pendingTasks">0</span>
        </span>
    </div>

    <!-- Task List Section -->
    <div class="task-list-container">
        <ul id="taskList" class="task-list">
            <!-- Tasks will be added here dynamically -->
        </ul>
        <div id="emptyMessage" class="empty-message">
            No tasks yet. Add one to get started! 🚀
        </div>
    </div>

    <!-- Clear All Button -->
    <div class="action-buttons">
        <button id="clearAllBtn" class="clear-all-btn">Clear All Tasks</button>
    </div>
</div>

<script src="script.js"></script>
```

```
</body>
</html>
```

❖ **CSS Code :**

```
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

body {
    font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    min-height: 100vh;
    display: flex;
    justify-content: center;
    align-items: center;
    padding: 20px;
}

.container {
    background-color: white;
    border-radius: 12px;
    box-shadow: 0 10px 30px rgba(0, 0, 0, 0.2);
    padding: 30px;
    max-width: 600px;
    width: 100%;
}
```

```
/* Header Styles */
.header {
    text-align: center;
    margin-bottom: 30px;
}

.header h1 {
    color: #333;
    font-size: 2.5rem;
    margin-bottom: 10px;
}

.description {
    color: #666;
    font-size: 1rem;
    line-height: 1.5;
}

/* Input Section Styles */
.input-section {
    display: flex;
    gap: 10px;
    margin-bottom: 25px;
}

.task-input {
    flex: 1;
    padding: 12px 15px;
```

```
    font-size: 1rem;
    border: 2px solid #e0e0e0;
    border-radius: 8px;
    transition: border-color 0.3s ease;
    outline: none;
}

.task-input:focus {
    border-color: #667eea;
    box-shadow: 0 0 8px rgba(102, 126, 234, 0.3);
}

.task-input::placeholder {
    color: #999;
}

.add-btn {
    padding: 12px 30px;
    background-color: #667eea;
    color: white;
    border: none;
    border-radius: 8px;
    font-size: 1rem;
    font-weight: 600;
    cursor: pointer;
    transition: background-color 0.3s ease, transform 0.2s ease;
}
```

```
.add-btn:hover {
    background-color: #5568d3;
}

.add-btn:active {
    transform: scale(0.98);
}

/* Task Counter Section */
.task-counter {
    display: flex;
    justify-content: space-around;
    background-color: #f5f5f5;
    padding: 15px;
    border-radius: 8px;
    margin-bottom: 20px;
    font-size: 0.95rem;
}

.counter-item {
    text-align: center;
    color: #333;
}

.counter-item strong {
    color: #667eea;
    display: block;
    font-size: 0.85rem;
```

```
    margin-bottom: 5px;
}

/* Task List Container */
.task-list-container {
    min-height: 200px;
    margin-bottom: 20px;
}

.task-list {
    list-style: none;
}

.task-item {
    display: flex;
    align-items: center;
    padding: 15px;
    background-color: #f9f9f9;
    border: 1px solid #e0e0e0;
    border-radius: 8px;
    margin-bottom: 12px;
    transition: all 0.3s ease;
}

.task-item:hover {
    background-color: #f0f0f0;
    box-shadow: 0 2px 8px rgba(0, 0, 0, 0.1);
}
```

```
.task-item.completed {
    background-color: #e8f5e9;
    border-color: #4caf50;
}

/* Checkbox Styles */
.task-checkbox {
    width: 20px;
    height: 20px;
    margin-right: 15px;
    cursor: pointer;
    accent-color: #667eea;
}

/* Task Text Styles */
.task-text {
    flex: 1;
    font-size: 1rem;
    color: #333;
    transition: all 0.3s ease;
}

.task-item.completed .task-text {
    color: #666;
    text-decoration: line-through;
}
```

```
/* Edit Input (hidden by default) */
.edit-input {
    flex: 1;
    padding: 8px 12px;
    font-size: 1rem;
    border: 2px solid #667eea;
    border-radius: 6px;
    outline: none;
}

/* Task Buttons Container */
.task-buttons {
    display: flex;
    gap: 8px;
    margin-left: 10px;
}

.edit-btn,
.delete-btn,
.save-btn,
.cancel-btn {
    padding: 6px 12px;
    border: none;
    border-radius: 6px;
    font-size: 0.85rem;
    font-weight: 600;
    cursor: pointer;
    transition: all 0.3s ease;
```

```
}

.edit-btn {
    background-color: #ff9800;
    color: white;
}

.edit-btn:hover {
    background-color: #f57c00;
}

.delete-btn {
    background-color: #f44336;
    color: white;
}

.delete-btn:hover {
    background-color: #da190b;
}

.save-btn {
    background-color: #4caf50;
    color: white;
}

.save-btn:hover {
    background-color: #45a049;
}
```

```
.cancel-btn {
    background-color: #9e9e9e;
    color: white;
}

.cancel-btn:hover {
    background-color: #757575;
}

/* Empty Message */
.empty-message {
    text-align: center;
    color: #999;
    padding: 40px 20px;
    font-size: 1.1rem;
}

/* Action Buttons */
.action-buttons {
    display: flex;
    justify-content: center;
    gap: 10px;
}

.clear-all-btn {
    padding: 10px 20px;
    background-color: #f44336;
```

```
    color: white;
    border: none;
    border-radius: 8px;
    font-size: 0.95rem;
    font-weight: 600;
    cursor: pointer;
    transition: background-color 0.3s ease;
}

.clear-all-btn:hover {
    background-color: #da190b;
}

.clear-all-btn:disabled {
    background-color: #ccc;
    cursor: not-allowed;
}

/* Responsive Design */
@media (max-width: 600px) {
    .container {
        padding: 20px;
    }

    .header h1 {
        font-size: 1.8rem;
    }
```

```
    .input-section {
        flex-direction: column;
    }

    .add-btn {
        width: 100%;
    }

    .task-counter {
        flex-direction: column;
        gap: 10px;
    }

    .task-buttons {
        flex-wrap: wrap;
    }

    .task-item {
        flex-direction: column;
        align-items: flex-start;
    }

    .task-buttons {
        width: 100%;
        margin-left: 0;
        margin-top: 10px;
    }
}
```

```
/* Animation for task addition */
@keyframes slideIn {
    from {
        opacity: 0;
        transform: translateY(-10px);
    }
    to {
        opacity: 1;
        transform: translateY(0);
    }
}

.task-item {
    animation: slideIn 0.3s ease;
}
```

❖**Javascript Code :**

```
const taskInput = document.getElementById('taskInput');
const dueInput = document.getElementById('dueInput');
const addBtn = document.getElementById('addBtn');
const taskList = document.getElementById('taskList');
const emptyMessage = document.getElementById('emptyMessage');
const clearAllBtn = document.getElementById('clearAllBtn');
const totalTasksSpan = document.getElementById('totalTasks');
const completedTasksSpan = document.getElementById('completedTasks');
const pendingTasksSpan = document.getElementById('pendingTasks');

// Tasks array to store all tasks
let tasks = [];
```

```
// Load tasks from localStorage on page load
document.addEventListener('DOMContentLoaded', () => {
   loadTasksFromStorage();
   renderTasks();
   updateTaskCounter();
});

// Event Listeners
addBtn.addEventListener('click', addTask);
taskInput.addEventListener('keypress', (e) => {
   if (e.key === 'Enter') {
      addTask();
   }
});
clearAllBtn.addEventListener('click', clearAllTasks);

/* ============================================
  TASK 4: Add & Display Tasks (JavaScript)
  ============================================ */

/**
 * Add a new task to the list
 * - Validates input (prevents empty tasks)
 * - Adds task to tasks array
 * - Saves to localStorage
 * - Re-renders the task list
 */
function addTask() {
   const taskText = taskInput.value.trim();
   const dueValue = dueInput.value; // datetime-local value
```

```
// Validate: Prevent empty tasks
if (taskText === '') {
    alert('Please enter a task!');
    taskInput.focus();
    return;
}

// Create task object
const task = {
    id: Date.now(),
    text: taskText,
    completed: false,
    createdAt: new Date().toLocaleString(),
    due: dueValue ? new Date(dueValue).toISOString() : null
};

// Add task to array
tasks.push(task);

// Save to localStorage
saveTasksToStorage();

// Clear input field
taskInput.value = '';
taskInput.focus();

// Re-render tasks
renderTasks();
updateTaskCounter();
```

```
  // Send task to backend API to schedule email reminder (if backend running)
  try {
    fetch('http://localhost:3000/api/tasks', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({ text: task.text, due: task.due })
    }).then(res => {
      if (res.ok) {
        console.log('Scheduled reminder on server');
      } else {
        console.warn('Server responded with', res.status);
      }
    }).catch(err => {
      console.warn('Could not reach reminder server:', err.message);
    });
  } catch (err) {
    console.warn('Error sending to server:', err.message);
  }
}

/* ============================================
  RENDER & DISPLAY TASKS
  ============================================ */

/**
 * Render all tasks to the DOM
 * - Clears previous list
 * - Creates HTML for each task
 * - Adds event listeners to buttons
 */
function renderTasks() {
```

```
    taskList.innerHTML = '';

    if (tasks.length === 0) {
        emptyMessage.style.display = 'block';
        return;
    }

    emptyMessage.style.display = 'none';

    tasks.forEach((task) => {
        const taskItem = createTaskElement(task);
        taskList.appendChild(taskItem);
    });
}

/**
 * Create a task element with all functionality
 * @param {Object} task - Task object
 * @returns {HTMLElement} - Task list item element
 */
function createTaskElement(task) {
    const li = document.createElement('li');
    li.className = `task-item ${task.completed ? 'completed' : ''}`;
    li.dataset.taskId = task.id;

    // Checkbox
    const checkbox = document.createElement('input');
    checkbox.type = 'checkbox';
    checkbox.className = 'task-checkbox';
    checkbox.checked = task.completed;
    checkbox.addEventListener('change', () => toggleTaskCompletion(task.id));
```

```
    // Task Text
    const taskText = document.createElement('span');
    taskText.className = 'task-text';
    taskText.textContent = task.text;

    // Buttons Container
    const buttonsDiv = document.createElement('div');
    buttonsDiv.className = 'task-buttons';

    // Edit Button
    const editBtn = document.createElement('button');
    editBtn.className = 'edit-btn';
    editBtn.textContent = 'Edit';
    editBtn.addEventListener('click', () => editTask(task.id));

    // Delete Button
    const deleteBtn = document.createElement('button');
    deleteBtn.className = 'delete-btn';
    deleteBtn.textContent = 'Delete';
    deleteBtn.addEventListener('click', () => deleteTask(task.id));

    buttonsDiv.appendChild(editBtn);
    buttonsDiv.appendChild(deleteBtn);

    li.appendChild(checkbox);
    li.appendChild(taskText);
    li.appendChild(buttonsDiv);

    return li;
}
```

```
/* ==========================================
   TASK 5: Edit & Delete Tasks
   ========================================== */

/**
 * Edit a task
 * - Replace task text with input field
 * - Add Save and Cancel buttons
 * @param {number} taskId - ID of task to edit
 */
function editTask(taskId) {
    const taskItem = document.querySelector(`[data-task-id="${taskId}"]`);
    const task = tasks.find((t) => t.id === taskId);

    if (!task) return;

    const taskText = taskItem.querySelector('.task-text');
    const buttonsDiv = taskItem.querySelector('.task-buttons');

    // Create edit input
    const editInput = document.createElement('input');
    editInput.type = 'text';
    editInput.className = 'edit-input';
    editInput.value = task.text;

    // Create Save button
    const saveBtn = document.createElement('button');
    saveBtn.className = 'save-btn';
    saveBtn.textContent = 'Save';
    saveBtn.addEventListener('click', () => saveEdit(taskId, editInput.value));
```

```
    // Create Cancel button
    const cancelBtn = document.createElement('button');
    cancelBtn.className = 'cancel-btn';
    cancelBtn.textContent = 'Cancel';
    cancelBtn.addEventListener('click', () => cancelEdit(taskId));

    // Replace task text with input
    taskText.replaceWith(editInput);
    buttonsDiv.innerHTML = '';
    buttonsDiv.appendChild(saveBtn);
    buttonsDiv.appendChild(cancelBtn);

    editInput.focus();
    editInput.select();
}

/**
 * Save the edited task
 * @param {number} taskId - ID of task to save
 * @param {string} newText - New task text
 */
function saveEdit(taskId, newText) {
    const newTextTrimmed = newText.trim();

    if (newTextTrimmed === '') {
        alert('Task cannot be empty!');
        return;
    }

    const task = tasks.find((t) => t.id === taskId);
```

```
    if (task) {
        task.text = newTextTrimmed;
        saveTasksToStorage();
        renderTasks();
        updateTaskCounter();
    }
}

/**
 * Cancel edit operation
 * @param {number} taskId - ID of task to cancel editing
 */
function cancelEdit(taskId) {
    renderTasks();
}

/**
 * Delete a task
 * @param {number} taskId - ID of task to delete
 */
function deleteTask(taskId) {
    const confirmDelete = confirm('Are you sure you want to delete this task?');

    if (!confirmDelete) return;

    tasks = tasks.filter((task) => task.id !== taskId);
    saveTasksToStorage();
    renderTasks();
    updateTaskCounter();
}
```

```
/* ============================================
   TASK 6 (BONUS): Task Completion Status
   ============================================ */

/**
 * Toggle task completion status
 * @param {number} taskId - ID of task to toggle
 */
function toggleTaskCompletion(taskId) {
    const task = tasks.find((t) => t.id === taskId);
    if (task) {
        task.completed = !task.completed;
        saveTasksToStorage();
        renderTasks();
        updateTaskCounter();
    }
}

/**
 * Update the task counter display
 * Shows total, completed, and pending tasks
 */
function updateTaskCounter() {
    const total = tasks.length;
    const completed = tasks.filter((task) => task.completed).length;
    const pending = total - completed;

    totalTasksSpan.textContent = total;
    completedTasksSpan.textContent = completed;
    pendingTasksSpan.textContent = pending;
```

```
    // Disable clear button if no tasks
    clearAllBtn.disabled = total === 0;
}

/* ============================================
   CLEAR ALL TASKS
   ============================================ */

/**
 * Clear all tasks after confirmation
 */
function clearAllTasks() {
    if (tasks.length === 0) return;

    const confirmClear = confirm(
        `Are you sure you want to delete all ${tasks.length} tasks? This action cannot be undone.`
    );

    if (!confirmClear) return;

    tasks = [];
    saveTasksToStorage();
    renderTasks();
    updateTaskCounter();
}

/* ============================================
   LOCAL STORAGE MANAGEMENT
   ============================================ */

/**
```

```
 * Save tasks to localStorage
 */
function saveTasksToStorage() {
   localStorage.setItem('todoTasks', JSON.stringify(tasks));
}

/**
 * Load tasks from localStorage
 */
function loadTasksFromStorage() {
   const savedTasks = localStorage.getItem('todoTasks');
   if (savedTasks) {
      try {
         tasks = JSON.parse(savedTasks);
      } catch (error) {
         console.error('Error loading tasks from storage:', error);
         tasks = [];
      }
   }
}
```

❖**Server :**

```
require('dotenv').config();
const express = require('express');
const cors = require('cors');
const nodemailer = require('nodemailer');

const app = express();
app.use(cors());
app.use(express.json());

const SMTP_HOST = process.env.SMTP_HOST;
```

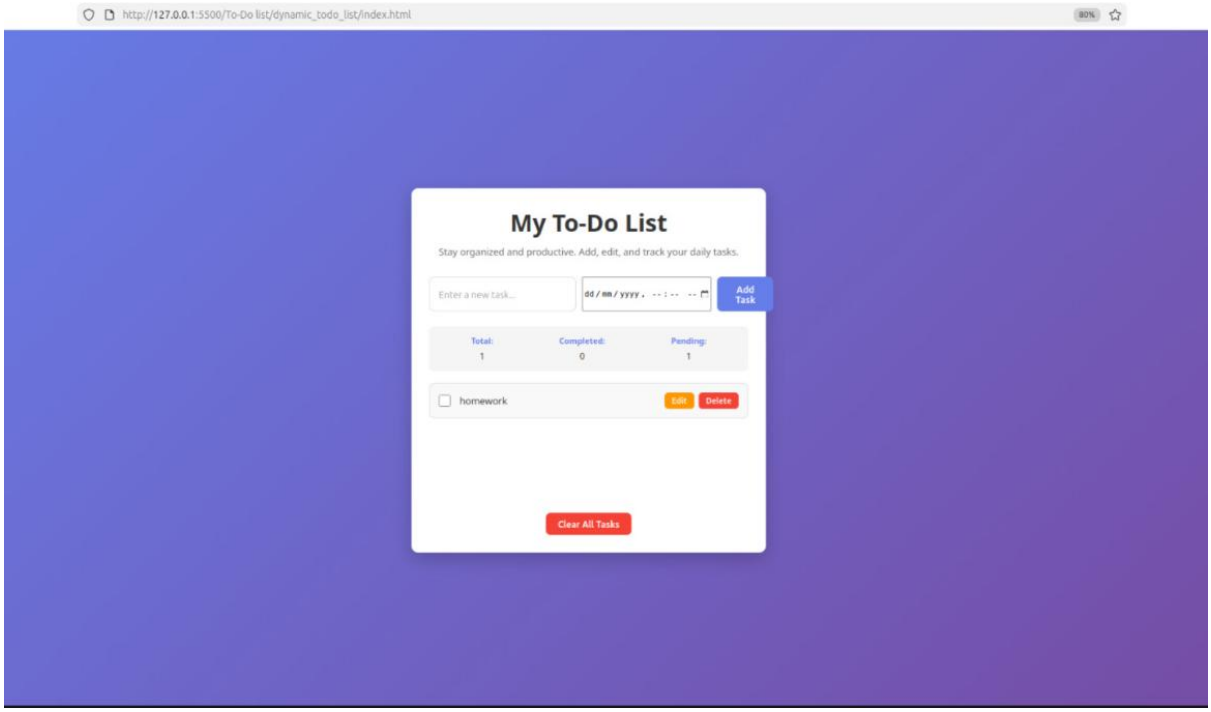
```
const SMTP_PORT = process.env.SMTP_PORT || 587;
const SMTP_USER = process.env.SMTP_USER;
const SMTP_PASS = process.env.SMTP_PASS;
const RECIPIENT_EMAIL = process.env.RECIPIENT_EMAIL;
const REMINDER_MINUTES = parseInt(process.env.REMINDER_MINUTES || '15', 10);

if (!SMTP_HOST || !SMTP_USER || !SMTP_PASS || !RECIPIENT_EMAIL) {
  console.warn('SMTP credentials or RECIPIENT_EMAIL not fully configured. See .env.example');
}

let transporter;
async function createTransporter() {
  if (SMTP_HOST && SMTP_USER && SMTP_PASS) {
    return nodemailer.createTransport({
      host: SMTP_HOST,
      port: Number(SMTP_PORT),
      secure: Number(SMTP_PORT) === 465,
      auth: { user: SMTP_USER, pass: SMTP_PASS }
    });
  }

  // Fallback: create a test account (Ethereal) for development/testing
  const testAccount = await nodemailer.createTestAccount();
  console.log('Using Nodemailer test account. Messages will be available at preview URLs.');
  return nodemailer.createTransport({
    host: 'smtp.ethereal.email',
    port: 587,
    auth: { user: testAccount.user, pass: testAccount.pass }
  });
}
```

```
(async () => { transporter = await createTransporter(); })();

// Simple in-memory store of scheduled timers (not persisted)
const scheduled = new Map();

app.get('/', (req, res) => {
  res.send('To-Do Reminder Server running');
});

// POST /api/tasks { text, due }
app.post('/api/tasks', (req, res) => {
  const { text, due } = req.body;

  if (!text) return res.status(400).json({ error: 'Task text required' });

  if (!due) {
    // If no due time provided, send immediate notification
    sendReminderEmail(text, new Date())
      .then(() => res.json({ status: 'sent_immediate' }))
      .catch(err => res.status(500).json({ error: err.message }));
    return;
  }

  const dueDate = new Date(due);
  if (isNaN(dueDate)) return res.status(400).json({ error: 'Invalid due date' });

  const reminderTime = new Date(dueDate.getTime() - REMINDER_MINUTES * 60000);
  const delay = reminderTime.getTime() - Date.now();

  const id = Date.now() + Math.random().toString(36).slice(2, 8);
```

```
  if (delay <= 0) {
    // reminder time passed — send immediately
    sendReminderEmail(text, dueDate)
      .then(() => res.json({ status: 'sent_immediate' }))
      .catch(err => res.status(500).json({ error: err.message }));
    return;
  }

  const timer = setTimeout(() => {
    sendReminderEmail(text, dueDate)
      .catch(err => console.error('Failed to send reminder:', err.message));
    scheduled.delete(id);
  }, delay);

  scheduled.set(id, { text, due: dueDate.toISOString(), timer });

  res.json({ status: 'scheduled', id, sendAt: reminderTime.toISOString() });
});

async function sendReminderEmail(taskText, dueDate) {
  if (!transporter || !RECIPIENT_EMAIL) return Promise.reject(new Error('Email not configured'));

  const formattedDue = dueDate ? new Date(dueDate).toLocaleString() : 'soon';
  const mailOptions = {
    from: SMTP_USER || 'no-reply@example.com',
    to: RECIPIENT_EMAIL,
    subject: `Reminder: upcoming task — ${taskText}`,
    text: `Reminder: Your task "${taskText}" is due at ${formattedDue}. This is an automated reminder
sent ${REMINDER_MINUTES} minutes before.`
  };
```

```
  try {
    const info = await transporter.sendMail(mailOptions);
    // Log preview URL for test accounts (Ethereal)
    const preview = nodemailer.getTestMessageUrl(info);
    if (preview) console.log('Preview URL:', preview);
    return info;
  } catch (err) {
    return Promise.reject(err);
  }
}

// end of file

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => {
  console.log(`Reminder server listening on port ${PORT}`);
});
```

❖**Output :**


http://127.0.0.1:5500/To-Do list/dynamic_todo_list/index.html
80%
My To-Do List
Stay organized and productive. Add, edit, and track your daily tasks.
Enter a new task...
dd / mm / yyyy , -- : -- --
Add Task
Total:
1
Completed:
0
Pending:
1
homework
Edit
Delete
Clear All Tasks